Assignment 1: Build a Single AI Agent with a Workflow

Trainer: Umashankar K H · Client: Innomatics

Pick a business problem, design one AI agent around it, draw its workflow, build it with the tools below, and prove it works with proper monitoring. You will get one of four sets (A, B, C or D). Each set has its own story, data and rules, but the steps are the same.

Before you start

What you must hand in

1. **Problem understanding**: half a page on who uses the agent, what goes in and what must come out.
2. **Workflow diagram**: every step, decision, tool call and the point where a human takes over. draw.io, Excalidraw or Mermaid is fine.
3. **Working agent**: one agent with at least four tools, built in LangGraph (plain LangChain is also accepted).
4. **results.csv**: one row per input record with the decision, the reason and the message the agent wrote.
5. **Monitoring and observability**: see the last section of this document.
6. **README plus a 3 to 5 minute screen recording** of one full run.

Ground rules

- One agent with many tools. A team of agents comes later, in Assignment 3.
- The LLM understands, extracts and writes. The business rules live in plain Python tools, so the answer is the same every time.
- Use temperature 0 for decisions. Keep API keys in a .env file, never in GitHub.
- Run on the full dataset, not only the first five rows.

Tools you can use

What you need	Easy choice	Good to know
LLM, free to start	NVIDIA Nemotron on build.nvidia.com. Use `nvidia/nemotron-3-nano-30b-a3b` (small, fast) or `nvidia/nemotron-3-super-120b-a12b` (bigger, stronger tool calling)	OpenAI-compatible API at `https://integrate.api.nvidia.com/v1` with your `NVIDIA_API_KEY`. Model names change, so check the catalogue
Embeddings (stretch goal only)	NVIDIA embedding model `nvidia/llama-3.2-nv-embedqa-1b-v2` through `NVIDIAEmbeddings`	`pip install langchain-nvidia-ai-endpoints`
Azure	Azure OpenAI or AI Foundry for the model, Azure Container Apps or Functions to host, Application Insights for logs	`AzureChatOpenAI` reads `AZURE_OPENAI_ENDPOINT` and `AZURE_OPENAI_API_KEY`
OpenRouter	`https://openrouter.ai/api/v1` with your OpenRouter key	A handy backup when one provider rate-limits you
Framework	LangChain for tools and prompts, LangGraph for the workflow	`pip install langchain langchain-openai langgraph`

Connecting to Nemotron takes four lines:

```
import os
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model='nvidia/nemotron-3-nano-30b-a3b',   # check build.nvidia.com for
the current name
    base_url='https://integrate.api.nvidia.com/v1',
    api_key=os.environ['NVIDIA_API_KEY'],
    temperature=0,
)
```

Prefer Azure? Swap in this instead:

```
from langchain_openai import AzureChatOpenAI
llm = AzureChatOpenAI(azure_deployment='<your-deployment>',
api_version='<api-version>', temperature=0)
```

The easy path in six steps

1. Turn the rules into a small decision table on paper before writing any code.
2. Write each tool as a normal Python function and test it with no LLM involved.
3. Wrap the functions with `@tool` and attach them to the model using `llm.bind_tools(tools)`.
4. Build a LangGraph graph: read the record, let the agent pick a tool, run the tool, decide, write the result. Loop until the agent has everything it needs.
5. Try five records and compare with your decision table. Fix, then repeat.
6. Switch on tracing, run all records and build the dashboard.

Set A: ClaimSure insurance claim triage agent

The story. ClaimSure is a mid-size insurance company. Every morning the claims desk has a pile of pending claims, and an adjuster spends about 15 minutes on each one: find the policy, check the rules, work out the payout, write to the customer. Management wants one agent that does the first pass on every claim, so humans only touch the tricky ones.

Your job. Build one agent that takes a claim ID and returns a decision, a payout amount, a short reason, and a friendly message for the customer. For claims that need a human, it must also open a review ticket with a three-line risk summary for the adjuster.

Data you get

`policies.csv` has 30 rows and `claims.csv` has 40 rows.

policy_id	customer_name	plan	coverage_limit	deductible	status	claims_last_12m	policy_age_days
POL1001	Bhavana Patil	Premium	800000	20000	Lapsed	3	1500
POL1002	Vikram Sharma	Basic	100000	5000	Active	1	200
POL1003	Vikram Gowda	Standard	300000	10000	Active	5	20

claim_id	policy_id	claim_type	amount_claimed	documents_attached	description
CLM5001	POL1025	Travel	340000	N	Checked-in baggage lost on international flight
CLM5002	POL1001	Travel	120000	Y	Medical emergency abroad
CLM5003	POL1005	Travel	900000	Y	Medical emergency abroad

All amounts are in rupees.

Business rules

Apply them **in this order**. The first rule that matches decides the outcome.

1. Policy status is not Active: **REJECT**, payout 0.
2. `documents_attached` is N: **REQUEST_DOCUMENTS**, payout 0.
3. `amount_claimed` is above 200,000, or `claims_last_12m` is 4 or more, or `policy_age_days` is below 30: **ESCALATE_MANUAL_REVIEW**, payout 0.
4. Otherwise **APPROVE**, with payout equal to the smaller of (amount claimed minus deductible) and the coverage limit. Payout is never below 0.

Tools your agent needs

Tool	What it does
`get_claim(claim_id)`	Returns the claim row
`get_policy(policy_id)`	Returns the policy row
`apply_rules(claim, policy)`	Applies the four rules in order and returns the decision plus which rule fired. No LLM inside
`calculate_payout(claim, policy)`	Does the payout maths
`create_review_ticket(claim_id, summary)`	Writes a ticket to `tickets.json` for escalated claims
`log_decision(...)`	Appends a row to `results.csv`

The LLM's own work is to read the claim description, decide which tools to call, write the customer message (friendly, under 80 words, says why) and write the adjuster's risk summary.

Hints

- A policy ID in a claim may not exist in `policies.csv`. Decide what the agent should do and write it in your README.
- Start with `apply_rules` alone and print the decision for all 40 claims before you add any LLM.

- Ask the model for structured output (a Pydantic model with decision, payout, reason, message) so `results.csv` is clean.
- Your output should contain all four decision types. If every row says APPROVE, a rule is not firing.

Stretch goal

Write ten short policy clauses in a text file, embed them with the NVIDIA embedding model, and let the agent quote the most relevant clause in its message to the customer.

Set B: ShopEasy returns and refund agent

The story. ShopEasy is an online store. After every sale, the support team is flooded with return requests, and most are routine. Some customers misuse returns, some ask after the window closed, and some ask for returns on orders that have not even arrived. Support wants one agent to handle the first response to every request.

Your job. Build one agent that takes a return request ID and decides what to do, then issues the refund (mock) or writes the right message to the customer.

Data you get

`customers.csv` has 30 rows, `orders.csv` has 50 rows and `return_requests.csv` has 40 rows.

customer_id	name	tier	returns_last_90d
CUS201	Rohan Sharma	Regular	3
CUS202	Naveen Sharma	Regular	1
CUS203	Revathi Kumar	Regular	0

order_id	customer_id	category	price_inr	delivery_status	days_since_delivery	payment_mode
ORD7001	CUS201	Beauty	1499	Delivered	9	Wallet
ORD7002	CUS214	Fashion	2499	Delivered	22	Card
ORD7003	CUS201	Beauty	1499	Delivered	15	Card

request_id	order_id	reason_text	has_photo
RET9001	ORD7048	Product is damaged on arrival	Y
RET9002	ORD7030	Size does not fit	N
RET9003	ORD7010	Quality is not as shown	Y

Return windows

Category	Days allowed after delivery
Electronics	7
Fashion	15
Home	10
Beauty	7
Grocery	Not returnable, see rule 2

Platinum customers get **5 extra days**. A return exactly on the last day is still allowed.

Business rules

Apply them **in this order**. The first rule that matches decides the outcome.

1. Order is not Delivered: **CANCEL_INSTEAD**. Tell the customer to cancel the order.
2. Category is Grocery: **REJECT**, unless the reason is damaged or wrong item **and** `has_photo` is Y, in which case **APPROVE_REFUND**.
3. `returns_last_90d` is 5 or more: **ESCALATE_ABUSE_REVIEW**.
4. `days_since_delivery` is above the window (plus 5 for Platinum): **REJECT_WINDOW_EXPIRED**.
5. Reason is damaged or wrong item and `has_photo` is N: **REQUEST_PHOTO**.
6. Otherwise **APPROVE_REFUND** for `price_inr`, back to the original payment mode. For cash on delivery, ask for bank transfer details, and never ask for card numbers or OTPs.

A reason counts as damaged or wrong item only when the customer says the product arrived damaged or the item is not the one ordered. Size, change of mind, quality, stopped working and late delivery never need a photo.

Tools your agent needs

Tool	What it does
`get_request(request_id)`, `get_order(order_id)`, `get_customer(customer_id)`	Plain lookups
`classify_reason(text)`	Uses the LLM to label the reason as damaged, wrong_item, size_fit, change_of_mind, quality_mismatch, defect_after_use or late_delivery
`apply_return_rules(order, customer, reason_label, has_photo)`	Applies the six rules and returns the decision and the rule number. No LLM inside
`issue_refund(order_id, amount, mode)`	Mock refund, appends to `refunds.csv`
`send_customer_message(request_id, text)`	Mock email, appends to `outbox.csv`

Hints

- The two steps that need language understanding are classifying the reason and writing the message. Everything else is code.
- Write ten hand-made test cases for `apply_return_rules` before you connect the LLM. Include a Platinum customer exactly one day over the window.
- Customers often reason in odd ways, for example 'arrived broken'. Test your classifier on a few paraphrases.
- Your output should contain several different decisions. If you see only APPROVE_REFUND, check rules 1 to 4.

Stretch goal

Write a one-page return policy FAQ, embed it with the NVIDIA embedding model, and let the agent quote the matching line in the rejection messages.

Set C: TechAssist IT helpdesk triage agent

The story. A company's IT helpdesk gets tickets written in every style: 'internet gone', 'clicked a weird link', 'laptop is dying'. Someone has to read each one, decide what kind of problem it is, how urgent it is and which team should take it. That someone is slow at 8 a.m. and the security tickets cannot wait.

Your job. Build one agent that reads each ticket, works out the category, priority, owning team and deadline, and writes the first reply to the employee.

Data you get

`tickets.csv` has 40 rows, `assets.csv` has 30 rows and `runbook.csv` has 7 rows.

ticket_id	asset_id	description	users_impacted
TKT4001	AST317	Laptop battery drains within 30 minutes	1
TKT4002	AST320	Antivirus flagged unknown file on my laptop	2
TKT4003	AST326	Received a suspicious email asking for my login, I clicked the link	2

asset_id	owner	os	warranty_status	department
AST301	Naveen Menon	Windows 10	Expired	Engineering
AST302	Bhavana Sharma	Windows 11	Expired	Sales
AST303	Rahul Gowda	Windows 11	In Warranty	Engineering

The runbook is the company's rulebook:

category	base_priority	assigned_team	sla_hours
PASSWORD	P4	Service Desk L1	8
VPN	P3	Network Team	8
HARDWARE	P3	Hardware Support	24
EMAIL	P3	Messaging Team	8

category	base_priority	assigned_team	sla_hours
SOFTWARE	P4	Service Desk L1	24
NETWORK	P2	Network Team	4
SECURITY	P1	Security Operations	1

Business rules

1. **Classify** the description into exactly one of the seven categories above. This is the LLM's main job.
2. **Look up** the runbook row for that category to get base priority, team and SLA hours.
3. **Raise the priority by one level** when `users_impacted` is 10 or more (P4 becomes P3, P3 becomes P2, P2 becomes P1). P1 stays P1.
4. If the category is HARDWARE and the asset's warranty is **Expired**, add the extra action **RAISE_PURCHASE_REQUEST**.
5. Write a **first response** to the employee. For P3 and P4 include two or three self-help steps. For SECURITY tell them to disconnect from the network and not to click anything else.

Tools your agent needs

Tool	What it does
`get_ticket(ticket_id)` and `lookup_asset(asset_id)`	Plain lookups
`classify_ticket(description)`	LLM call that returns one category, with structured output so it cannot invent a new one
`lookup_runbook(category)`	Returns base priority, team and SLA
`calculate_priority(base_priority, users_impacted)`	Applies the bump rule in code
`create_ticket_record(...)`	Writes the routing result to `routed_tickets.csv`

Tool	What it does
`notify_team(team, ticket_id, priority)`	Mock Slack message written to `notifications.log`

Hints

- Use a Pydantic model with a `Literal` type for the category. The model must pick from the seven values and nothing else.
- Ticket text is messy. Try a few paraphrases of your own ('cannot login', 'wifi dead in 3rd floor') to see whether the classifier stays steady at temperature 0.
- The priority bump is where agents often go wrong when the LLM does the maths. Keep it in `calculate_priority`.
- Your results should contain all four priority levels. If nothing is P1, check the SECURITY handling.

Stretch goal

Write a short knowledge base of fixes (reset password, reconnect VPN, clear Outlook cache), embed it with the NVIDIA embedding model and let the agent pick the best self-help steps by meaning.

Set D: FreightFlow shipment exception agent

The story. FreightFlow moves goods between eight Indian cities with five carriers. Every day some shipments run late, and the operations team must decide who to tell and what to do. A late box of furniture is an annoyance. A late box of vaccines is an emergency.

Your job. Build one agent that reviews each shipment, picks the right action, finds a better carrier when a reroute is needed, and writes the message to the customer.

Data you get

`shipments.csv` has 40 rows and `carriers.csv` has 5 rows.

shipment_id	origin	destination	carrier	delay_hours	cargo_type	perishable	value_inr	exception_note
SHP6001	Bengaluru	Kochi	SwiftHaul	8	Fresh Produce	Y	300000	Truck breakdown on highway

shipment_id	origin	destination	carrier	delay_hours	cargo_type	perishable	value_inr	exception_note
SHP6002	Hyderabad	Kochi	BlueDart-X	4	Pharma	Y	120000	Truck breakdown on highway
SHP6003	Chennai	Delhi	RoadRunner	20	Furniture	N	40000	Heavy rain, road blocked

carrier	reliability_score	cost_per_km_inr
BlueDart-X	0.95	18
SwiftHaul	0.91	14
RoadRunner	0.84	11
CargoKing	0.88	12
EcoMove	0.93	16

Business rules

Work out the main action first, then check the extra action.

1. `delay_hours` below 6: **NO_ACTION**.
2. `delay_hours` from 6 to 24 inclusive: **NOTIFY_CUSTOMER**.
3. `delay_hours` above 24 and `perishable` is N: **ESCALATE_OPS_MANAGER**.
4. `delay_hours` above 24 and `perishable` is Y: **CRITICAL_REROUTE**. Pick a carrier **different from the current one** with reliability of 0.90 or more and the **lowest cost per km**. Name the chosen carrier.
5. **Extra action:** if `value_inr` is above 500,000 and `delay_hours` is 6 or more, also do **NOTIFY_ACCOUNT_MANAGER**. A shipment can therefore have two actions.

Tools your agent needs

Tool	What it does
`get_shipment(shipment_id)` and `list_carriers()`	Plain lookups
`decide_actions(shipment)`	Applies rules 1 to 5 and returns the list of actions. No LLM inside

Tool	What it does
`pick_alternate_carrier(current_carrier, min_reliability)`	Returns the cheapest qualifying carrier that is not the current one
`draft_customer_update(shipment, exception_note)`	LLM writes a short, calm update in plain language
`alert_ops(shipment_id, severity)` and `notify_account_manager(shipment_id)`	Mock alerts written to `alerts.log`
`log_result(...)`	Appends a row to `results.csv`

Hints

- The customer message must use the exception note in everyday words, must not blame the carrier, and must not promise an exact arrival time.
- If the current carrier is itself the cheapest qualifying one, the reroute must choose the next cheapest. Write a test for that.
- Shipments with `delay_hours` of 0 still have a note in the file. The note alone is never a reason to act.
- Save the list of actions in `results.csv` as one text column, for example `ESCALATE_OPS_MANAGER+NOTIFY_ACCOUNT_MANAGER`.

Stretch goal

Add a tool that estimates the extra cost of a reroute from the distance between the two cities (make up a small distance table) and the carrier's cost per km, and have the agent mention it in the alert.

Monitoring, observability and marking

An agent you cannot see inside is an agent you cannot trust. Whatever set you got, add monitoring to the same agent and show it working.

What to record for every run

Signal	Why it matters
A trace per record: input, every tool call with its arguments and result, final decision	Lets you replay any wrong decision and see exactly where it went wrong
Latency per step and in total	Shows which tool or model call is slow
Tokens in and out per LLM call, and the estimated cost	Keeps the bill visible. Read the usage numbers the API returns
Errors and retries: tool failures, badly formed LLM output, timeouts	The early warning that something is breaking
The decision and the rule that fired	Lets you check the mix of outcomes against what you expect

How to do it, the easy way

- **Langfuse** (open source, free cloud tier): add its callback handler to your LangChain or LangGraph run and every step is traced.
- **LangSmith**: set the tracing environment variables and runs appear in its dashboard.
- **Azure**: send OpenTelemetry data to Application Insights using `azure-monitor-opentelemetry`, then build a Workbook.
- **Always** also write your own log file, so you are not tied to any one tool. One JSON line per step, like this:

```
{"run_id": "r-0007", "record_id": "CLM5007", "step": "tool:get_policy",
"latency_ms": 41, "tokens_in": 0, "tokens_out": 0, "status": "ok"}
```

The dashboard (one page)

Show at least: number of runs, success rate, average and 95th percentile latency, total tokens, estimated cost, tool error rate, and a bar chart of decisions by type. Langfuse, LangSmith, an Azure Workbook or a small Streamlit page are all fine.

Alerts (at least two)

Pick from these or invent your own: tool error rate above 5 percent in a run, 95th percentile latency above 20 seconds, a single run costing more than a budget you set, or a decision value that is not in the allowed list. Show how the alert fires (a log line, an email or a Teams or Slack webhook).

Prove it works

Break something on purpose, for example point one tool at a wrong file or use a bad API key, and show in your dashboard how the failure appears and which alert fires.

How you will be marked

Area	Marks
Problem understanding and workflow diagram	15
Tool design (clear, tested, no LLM where code is better)	20
Agent correctness on the full dataset	25
Guardrails and error handling	10
Monitoring and observability (traces, dashboard, alerts)	20
Code quality, README and demo	10
Total	**100**